

IFT436 - Série d'exercices #5 : les algorithmes gloutons

Manuel Lafond

Exercice 1: Dans une journée, un certain professeur X a t plages de rendez-vous disponibles, chacune pouvant être allouée à un étudiant qui nécessite un entretien. Chaque étudiant a une plage maximum possible, e.g. un étudiant demande un rendez-vous avant la plage i , avec $1 \leq i \leq t$. Il s'avère que X est une personne ignoble, et affecte une importance différente à chaque étudiant. Le but est d'affecter chacune des t plages de temps à un étudiant de façon à maximiser la somme des importances des étudiants traités.

Formellement, on a le problème suivant:

Problème du professeur discriminant

Entrée: un ensemble $E = \{(x_1, p_1), (x_2, p_2), \dots, (x_n, p_n)\}$, où chaque (x_i, p_i) représente un étudiant: x_i est le temps maximum possible pour un rendez-vous, et p_i est l'importance de l'étudiant.

Sortie: un tableau $T = [(x'_1, p'_1), \dots, (x'_t, p'_t)]$ comprenant des éléments distincts de E , où $T[i]$ est l'étudiant assigné au temps i . On peut permettre $T[i] = \text{null}$ si on n'affecte personne à la plage i . On exige que pour tout $i \in \{1, \dots, t\}$, si l'élément $T[i] = (x'_i, p'_i)$ est non-null, il satisfait $i \leq x_i$. On veut que $\sum_{i=1}^k p'_i$ soit maximum parmi tous les choix possibles.

Considérez l'algorithme glouton suivant qui insère les étudiants en ordre d'importance, lorsque c'est possible, le plus tard possible.

fonction

```
professeurDiscriminant( $E = \{(x_1, p_1), (x_2, p_2), \dots, (x_n, p_n)\}, t$ )
  Trier  $E$  selon les importances  $p_i$ , en ordre décroissant
   $T = [null] * t$ 
  pour  $i = 1$  à  $n$  faire
    //trouver l'endroit le plus tard possible pour  $E[i] = (x_i, p_i)$ 
    //On commence par essayer  $x_i$ , puis  $x_i - 1$ , etc.
    pour  $j = x_i$  à 1 faire
      si  $T[j] = null$  alors
         $T[j] = (x_i, p_i)$ 
        break // Sortir de la boucle pour  $j$ 
    fin
  fin
  return  $T$ 
```

Montrez que cet algorithme est optimal, c'est-à-dire qu'il maximise la somme des importances.

Solution.

Soit T la solution retournée par notre algorithme et T^* une solution optimale. Considérons l'ordre $(x'_1, p'_1), \dots, (x'_k, p'_k)$ dans lequel notre algorithme assigne les éléments dans T . Soit (x'_i, p'_i) le premier étudiant qui est assigné à une certaine place j , mais qui est soit non-assigné dans T^* ou bien qui n'est pas assigné à la même place (donc, pour $k \in \{1, \dots, i-1\}$, (x'_k, p'_k) est assigné au même endroit dans notre solution et dans T^*). Si un tel élément n'existe pas, nous avons fini car $T = T^*$ est optimal.

Donc on a $T[j] = (x'_i, p'_i)$ et $j \leq x'_i$. Si $T^*[j] = null$, on peut modifier T^* en mettant $T^*[j] = (x'_i, p'_i)$ (si cet étudiant (x'_i, p'_i) était déjà affecté dans T^* , ceci ne fait que le déplacer à un autre indice valide, et si (x'_i, p'_i) n'était affecté nulle part dans T^* alors on a une meilleure solution, ce qui serait une contradiction). On peut donc supposer que $T^*[j] \neq null$, et donc $T^*[j] = (x, p)$, pour un certain x et p . Puisque les éléments de T ajoutés avant (x'_i, p'_i) étaient dans T^* au même endroit, on sait que (x, p) ne fait pas partie de $(x'_1, p'_1), \dots, (x'_{i-1}, p'_{i-1})$. Ceci veut dire que (x, p) faisait partie des choix possibles pour notre algorithme lorsque (x'_i, p'_i) a été assigné. De plus, puisque nous considérons les étudiants en ordre décroissant d'importance, on doit avoir $p \leq p'_i$. Si on suppose que (x'_i, p'_i) n'est pas dans T^* , il s'ensuit qu'en remplaçant $T^*[j] = (x'_i, p'_i)$, on obtient une solution égale ou meilleure.

Par contre, il est possible que (x'_i, p'_i) soit déjà dans T^* , mais à une autre position. Puisque notre algorithme et T^* ont fait les mêmes choix jusqu'à (x'_i, p'_i) , et puisque nous choisissons l'emplacement le plus tard possible pour (x'_i, p'_i) à ce moment, on déduit que dans T^* , cet (x'_i, p'_i) est assigné avant la position j , disons à la position $k < j$. Mais alors, on peut interchanger (x'_i, p'_i) et (x, p) , i.e., on peut poser $T^*[j] = (x'_i, p'_i)$ (car $j \leq x'_i$) et $T[k] = (x, p)$ (qui fonctionne car $k < j$).

La valeur de la solution n'a pas changé. Par contre, le T^* original plaçait $i - 1$ éléments en commun avec notre solution, alors que le T^* modifié place i éléments en commun avec notre solution. On peut répéter cet argument autant de fois qu'on veut jusqu'à ce qu'on atteigne T tout en maintenant l'optimalité, et donc T est optimal.

Exercice 2: Supposons que l'on vit à une époque où les pièces de monnaie sont 25 sous, 10 sous, 5 sous et 1 sou. On doit rendre à un client un certain nombre de sous t en minimisant le nombre de pièces redonnées. Par exemple pour rendre 98 sous, on redonnerait $3 \times 25 + 2 \times 10 + 3 \times 1$ pièces. Plus généralement, on pourrait avoir un ensemble d'entiers P dénotant les valeurs de pièces disponibles (donc dans l'exemple, $P = \{25, 10, 5, 1\}$). L'algorithme glouton fait la chose suivante:

```

fonction minPieces( $t, P$ )  $t$  est la somme cible,  $P$  les valeurs de pièces
     $R = ()$ 
     $s = 0$ 
    tant que  $s < t$  faire
        Soit  $p$  la valeur maximum de  $P$  telle que  $s + p \leq t$ 
         $s = s + p$ 
        Ajouter  $p$  à  $R$ 
    fin
    return  $R$ 

```

Montrez que cet algorithme fonctionne correctement avec $P = \{25, 10, 5, 1\}$. Montrez ensuite qu'il ne fonctionne pas sur certaines entrées P .

Solution.

Soit $R = (r_1, \dots, r_k)$ la liste des pièces retournées par notre algorithme, et soit $S = (s_1, \dots, s_h)$ une solution optimale. On suppose que R et S sont triées en ordre décroissant. On note les choses suivantes sur S . Il n'y a pas plus de 2 pièces de 10 sous (car s'il y en a au moins 3, on les remplacerait par $25 + 5$); il n'y a pas plus de 1 pièce de 5 sous (sinon on en remplacerait 2 par 10 sous); et il n'y a pas plus de 4 pièces de 1 sous.

Soit r_i le premier élément de R qui diffère de S , i.e. $r_1 = s_1, \dots, r_{i-1} = s_{i-1}$ mais $r_i \neq s_i$. Si r_i n'existe pas, $R = S$ et nous avons fini. Soit

$$q = r_i + r_{i+1} + \dots + r_k = s_i + s_{i+1} + \dots + s_h$$

i.e. q est la somme restante à rembourser après les $i - 1$ premiers choix. Supposons d'abord que $r_i = 25$. Puisque r_i est la première différence, on a que $s_i \neq 25$. De plus, puisque l'on a fait un tri, on a $s_j \neq 25$ pour chaque j de i à h . De plus, $q \geq 25$. Il doit y avoir 2 pièces de 10 sous et 1 pièce de 5 sous dans $\{s_i, \dots, s_h\}$, car sinon il est impossible que $s_i + \dots + s_h \geq q$ (en se rappelant des nombres maximum de chaque pièce). Mais alors, on pourrait remplacer $2 \times 10 + 1 \times 5$ par une seule pièce de 25 sous, contre-disant l'optimalité de S . Supposons alors que $r_i = 10$. Ceci est aussi impossible: au maximum, $\{s_i, \dots, s_h\}$ contient 1 pièce de 5 sous et 4 pièces de 1 sou, alors que $q \geq 10$. Finalement, si on suppose que $r_i = 5$, on voit que $q \geq 5$ mais $\{s_i, \dots, s_h\}$ contient au maximum 4 pièces de 1 sou. Ces deux cas mènent aussi à une contradiction de l'optimalité de S .

Donc en fait, on vient de montrer que l'on produit la seule solution optimale possible, car toutes les autres façons de faire mènent à une contradiction.

Exercice 3: L'algorithme de Kruskal trouve un arbre couvrant minimum dans un graphe valué. Il n'est pas identique à l'algorithme de Prim, car il maintient une forêt (i.e. un ensemble d'arbres) au lieu d'un arbre partiel. À chaque étape, l'idée est d'ajouter l'arête de coût minimum qui ne crée pas de cycle dans la solution courante.

```

fonction algoDeKruskal( $G = (V, E, f)$ )
     $T = (V, \emptyset)$  // Notre sous-graphe partiel.
    tant que  $T$  a moins de  $n - 1$  arêtes faire
        Soit  $e$  l'arête de coût minimum telle que l'ajout de  $e$  à  $T$  ne
            créé pas de cycle
        Ajouter  $e$  à  $T$ 
    fin
    return  $T$ 

```

Montrez que l'algorithme de Kruskal trouve bel et bien un arbre couvrant minimum.

Solution.

La preuve est très similaire à celle de Prim. Soit $E' = \{e_1, \dots, e_{n-1}\}$ les arêtes ajoutées par l'algorithme de Kruskal, dans l'ordre, et soit T^* un arbre couvrant minimum. Soit $e_i = xy$ la première arête qui n'est pas dans T^* . Considérez le graphe T' obtenu en ajoutant e_i à T^* . On sait que T' a un cycle C qui contient e_i et le chemin de x à y dans T^* . De plus, il y a une arête uv de C qui n'est pas dans E' (sinon, E' aurait un cycle). Au moment où l'algorithme a choisi e_i , l'arête uv faisait donc partie des choix possibles, car l'ajout de uv à ce moment n'aurait pas créé de cycle (ceci est parce que T^* contient $e_1, \dots, e_{i-1}$ et a ajouté uv). Par le choix glouton de l'algorithme, $f(uv) \geq f(e_i)$. En retirant l'arête uv de T^* et en ajoutant e_i , on obtient un autre arbre couvrant de coût inférieur ou égal (c'est un arbre car le sous-graphe obtenu a $n - 1$ arêtes et est connexe parce qu'enlever une arête d'un cycle ne déconnecte pas un graphe).

On peut répéter cet argument jusqu'à l'obtention de E' , ce qui veut dire que l'arbre retourné est optimal.
